

LearnPath AI

AI-Powered Personalized Learning Path Recommender

Solution Documentation

Team: PathFinders | IILM University

Conversational goal capture · explainable hybrid recommendations · prerequisite-aware roadmap generation ·
adaptive progress dashboard

1. Problem Understanding

Online learning platforms offer thousands of courses, but learners struggle to identify the right *sequence* of resources to reach a specific goal. A one-size-fits-all catalog doesn't account for differing skill levels, prior experience, career aspirations, or available time. The core problem has three parts that a solution must solve together, not independently:

- **Understanding the learner** — extracting goals, known skills, experience level, and constraints from natural language, not a rigid form.
- **Identifying the skill gap** — knowing exactly which skills separate the learner's current state from their target role.
- **Sequencing correctly** — a list of relevant courses is not a path; prerequisites must be respected so a learner is never handed material they aren't ready for.

Critically, the brief also requires **explainability** and **adaptivity** — recommendations must be justified, not just produced, and the path must change in response to real learner feedback and progress, not stay static after generation.

2. Solution Approach

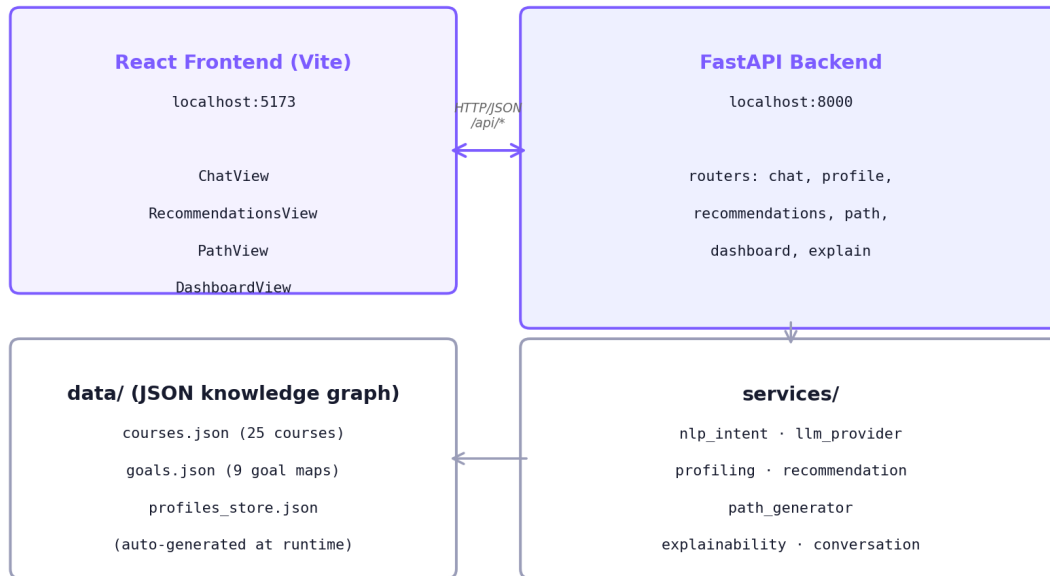
LearnPath AI is a full-stack web application with a conversational front door and four purpose-built engines behind it. Rather than a single black-box model, each requirement in the brief maps to a distinct, inspectable component — this keeps every recommendation traceable to a concrete reason, which is what explainability actually requires.

Requirement	Component	Approach
Conversational interface	ChatView + conversation.py	Multi-turn dialogue; rule-based NLU with an optional real-LLM upgrade hook
Learner profiling engine	profiling.py	Incremental signal merging into a persistent LearnerProfile
Recommendation engine	recommendation.py	Hybrid content-based scoring across 4 weighted signals
Path generator	path_generator.py	Topological sort (Kahn's algorithm) over a prerequisite DAG
Explainability + Q&A	explainability.py	Reasons attached to every score; rule-based intent matching for free-form ques
Progress dashboard	DashboardView.jsx	Live aggregation of completion %, skills, and milestone timeline

End-to-end flow: A learner describes their goal in chat → NLU extracts goal/skills/experience/hours → profiling engine persists it → recommendation engine scores the full course catalog against the skill gap → path generator sequences the relevant subset into a valid roadmap with milestones → marking a course complete (or flagging it too easy/hard) updates the profile and **regenerates the path live**.

3. System Architecture

A React (Vite) frontend communicates with a FastAPI backend over a JSON REST API, proxied through `/api/*` in development. The backend is organized into routers (HTTP layer) and services (business logic), backed by a small JSON knowledge graph of courses and career-goal templates.



Why this shape: every engine is a plain Python module with pure functions wherever possible (e.g. `score_course()`, `generate_path()`), which made it possible to unit-test the recommendation and path logic directly against fixture profiles, independent of the API or UI — this is how the bugs described in Section 6 were actually caught.

4. AI/ML Techniques Used

Rule-based + fuzzy-matching NLU

Goal/skill/experience-level/pace extraction from free text using pattern matching (regex) and sequence-similarity fuzzy matching (`difflib.SequenceMatcher`) for typo tolerance. Includes negation-window detection so phrasing like "new to X" or "no X yet" is not misread as a positive skill claim, and goal-phrase masking so role titles like "DevOps engineer" aren't misread as a claim to know "DevOps".

Hybrid content-based recommendation

Each course is scored as a weighted combination of: skill-gap coverage (50%), prerequisite readiness (25%), experience-level fit (15%), and a popularity prior standing in for a collaborative-filtering signal (10%). Every score decomposes into human-readable reasons — this is what makes recommendations explainable by construction rather than post-hoc.

Graph algorithm — topological sort

The course catalog is modeled as a prerequisite DAG (skills, not courses, are the dependency unit). Kahn's algorithm sequences the relevant subset into a roadmap where every prerequisite is guaranteed to appear before it's needed. Domain-dominance weighting prevents an out-of-domain course from leaking in just because it shares a generically-named skill tag with an in-domain one.

Explainable-by-construction design

Rather than a black-box relevance score, `contributing_factors` are the same reasons used internally to compute the score — the explanation is never a separate, potentially-inconsistent narrative bolted on afterward.

Pluggable LLM upgrade hook

The conversational layer runs fully offline by default (deterministic, free, reproducible for grading). `llm_provider.py` is a single integration seam where a real Claude/GPT API call can be enabled via environment variables, with automatic graceful fallback to the template engine on any failure — so the architecture supports a production-grade NLU upgrade without touching any other module.

5. Key Features & Workflows

5.1 Conversational goal capture — the learner states goal, known skills, and availability in plain language; the assistant confirms what it understood and asks only for what's missing.

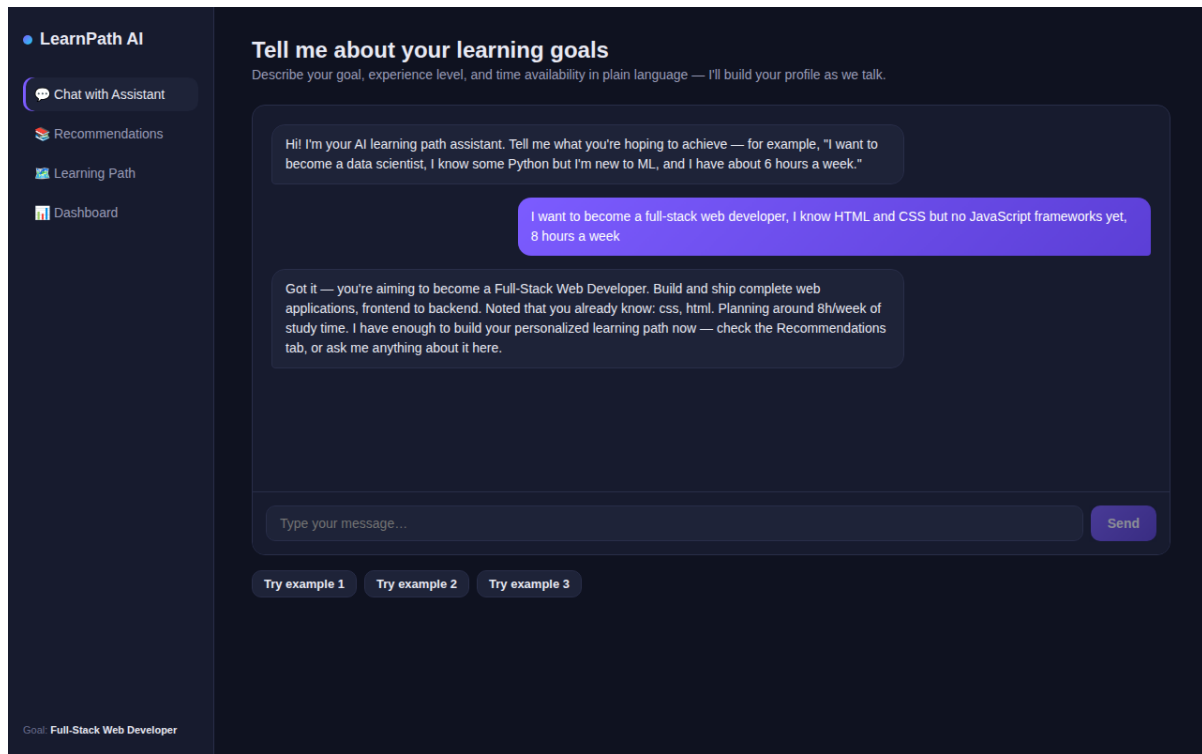


Fig 1. Chat interface after the learner states a goal — negation ('no JavaScript frameworks yet') is correctly excluded from known skills.

5.2 Explainable recommendations — every course shows a match score and the specific reasons behind it (skill-gap coverage, prerequisite readiness, popularity).

LearnPath AI

Chat with Assistant

Recommendations

Learning Path

Dashboard

Recommended for you

Ranked by skill-gap coverage, prerequisite readiness, level fit, and popularity. Tap "Why this?" for the reasoning.

HTML, CSS & JavaScript Basics

Web Development · beginner · course · 25h

Core building blocks of the web: markup, styling, and scripting.

- Teaches 1 skill(s) you still need for your goal: javascript
- Matches your current experience level (beginner)
- Highly rated / popular among learners with similar goals

[Why this?](#)[Open resource ↗](#)[Mark completed](#)

Git & GitHub Essentials

Programming · beginner · course · 8h

Version control fundamentals, branching, and collaborative workflows.

- Teaches 1 skill(s) you still need for your goal: git
- Matches your current experience level (beginner)

[Why this?](#)[Open resource ↗](#)[Mark completed](#)

React.js for Frontend Development

Web Development · intermediate · course · 30h

Component-based UI development with React, hooks, and state management.

- Teaches 2 skill(s) you still need for your goal: javascript, react
- Requires prerequisite skill(s) first: javascript
- Highly rated / popular among learners with similar goals

[Why this?](#)[Open resource ↗](#)[Mark completed](#)

Node.js & Express Backend Development

Web Development · intermediate · course · 30h

[50% match](#)

Fig 2. Ranked recommendations with inline, per-course reasoning.

5.3 Prerequisite-aware roadmap — courses are sequenced into milestones (Foundations → Core Skills → Applied Practice → Mastery), each step showing what it builds on.

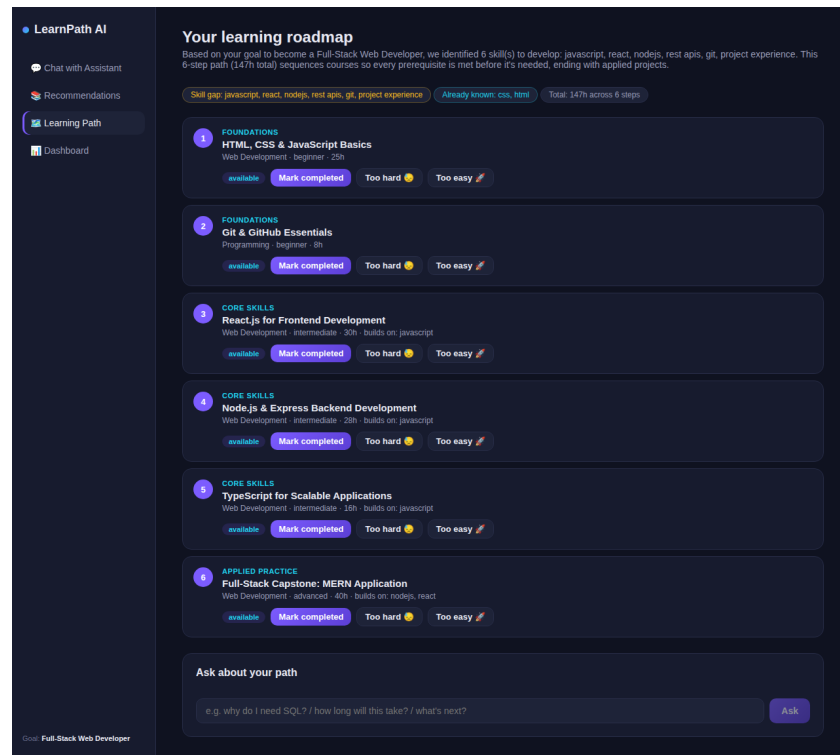


Fig 3. Generated roadmap for a Full-Stack Web Developer goal — a correctly domain-scoped, dependency-ordered path.

5.4 Adaptive progress dashboard — completion %, skills acquired/remaining, and hours-by-milestone update live as courses are completed.

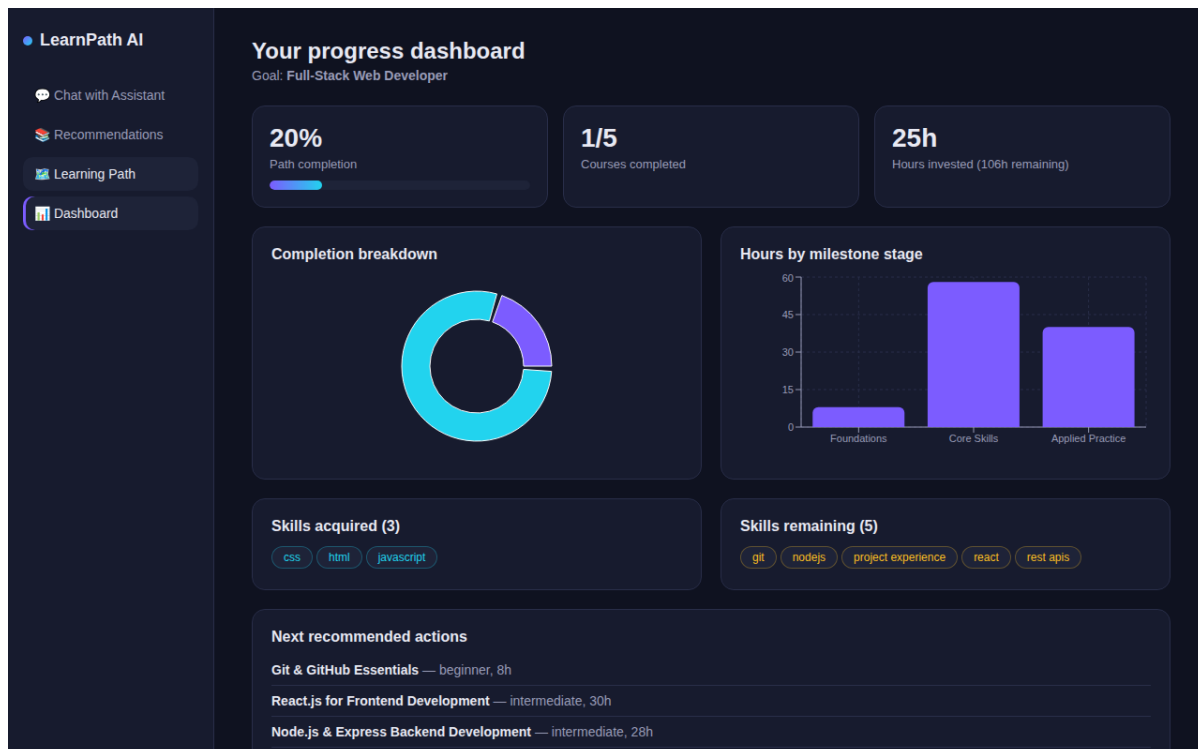


Fig 4. Dashboard after completing one course — the path and stats have already adapted.

6. Challenges Faced

Three real correctness bugs were found and fixed during development through direct testing against the running API (not just code review) — documenting them here because catching and fixing them was a meaningful part of the engineering effort, and being upfront about what broke and why is more useful than presenting a sanitized process.

Negation was not initially handled in skill extraction

"I'm new to machine learning" was initially extracted as the learner *knowing* machine learning, because the skill-matching regex only checked for the presence of the phrase, not its polarity. Fixed by scanning a lookback window before each skill match for negation cues ("new to", "no ... yet", "never used", etc.) before accepting it as a positive skill claim.

Goal role-names collided with catalog skill names

Several skill tags in the catalog (devops, react, nlp, mlops, machine-learning) are substrings of how the corresponding goal is naturally phrased ("DevOps engineer", "React developer"). Without correction, stating a goal alone would falsely register that skill as already known. Fixed by masking the exact character span matched by the goal keyword before running skill extraction over the remaining text.

Generic skill tags caused cross-domain leakage in the path generator

Two unrelated capstone courses (an ML capstone and a full-stack capstone) both taught a generic "project-experience" tag. The path generator's seed-selection step was domain-blind, so satisfying that one shared tag pulled in the wrong capstone plus its entire prerequisite chain (Python, Statistics, Machine Learning) into an otherwise correctly-scoped Web Development roadmap. Fixed by computing each goal's dominant domain and preferring in-domain course matches when a skill is taught in multiple domains.

Known scope limitations (by design, for a hackathon timeline): the 25-course catalog is a curated demo dataset rather than a live platform integration; persistence is a local JSON file rather than a production database; and the popularity-prior score is a static placeholder for what would be a real collaborative-filtering signal derived from enrollment/completion data at scale.